KRISHISYNC AI | VIBE CODING SERIES | 1 OF 5

Pitch and Master Prompt

Story, impact and the context prompt that starts every AI coding session

Market-Aware Agri-Coordination & Smart Energy Optimization for Indian Smallholders

Challenge 1: Sustainable Agriculture & Resource Optimization

Executive Pitch and Master Prompt

This document does two jobs. First, it is the one-page story of KrishiSync AI for judges and teammates. Second, it contains the **Master Prompt**: the context block you paste at the start of every AI coding session so the assistant always knows what you are building.

The problem: a dual crisis

Pressure	What the data says (from the project deck)
Workforce density	About 40% of the population is employed in highly fragmented agricultural operations.
Resource footprint	About 90% of freshwater is drawn through manual flood irrigation.
Grid inefficiency	Traditional rural pump sets create severe peak-hour stress on the grid.
Market shock	15-20% of crop value is lost after harvest.

The Copycat Trap. Smallholders cannot see what their neighbours are planting, so everyone plants the same crop at the same time. Supply floods the local mandi, prices collapse, and the farmer absorbs the loss. Existing tools have no regional-density view, so the problem stays invisible.

The solution: a connected farm ecosystem

KrishiSync AI is a unified web platform that links each farm to live market data and grid status. It has two engines and one planning layer:

- **Anti-Saturation Engine.** Colour-coded glut alerts when planned crop volume within a 20 km radius crosses a safe threshold, plus ranked alternatives.
- **Smart Energy Engine.** Weather-predictive pump scheduling, off-peak load balancing, PM-KUSUM solar integration, and idle-load tracking.
- **Terrain-aware Agro-Advisory.** A dynamic calendar driven by soil moisture, N-P-K levels and geomorphology.

Farmer journey, seed to market

Step	Name	What happens
01	Onboarding	Farmer registers land location and terrain profile.
02	Market check	Anti-glut algorithm recommends crops.
03	Dynamic calendar	Live seeding and watering alerts.
04	Smart grid sync	Heavy irrigation runs in off-peak hours.

Why it wins on the challenge

Challenge goal	KrishiSync answer
Reduce resource intensity	Automated weather-predictive pumping cycles.
Strengthen resilience	Terrain data helps prevent waterlogging and runoff.
Empower smallholders	High-margin, market-protected crop alternatives.

Impact targets

Vector	Metric targeted	System action
Economic	Stabilise seasonal income	Shield from oversupply; target 25% fertiliser cost reduction
Environmental	Prevent resource depletion	Avoid grid overload via automated smart solar pumping
Operational	Advance net-zero operations	Lower carbon intensity, curtail organic waste

The 25% figure is a target stated in the deck, not a measured result. Present it as a goal until pilot data exists.

Series map. 1 Pitch and Master Prompt (this file) | 2 Architecture | 3 UI/UX | 4 Algorithms | 5 Roadmap and Demo. Open the matching PDF when you start that part of the build and paste its prompts in order.

The Master Prompt

Paste this at the start of every new chat with your AI coding tool (Cursor, Claude, Copilot, Windsurf). It keeps the assistant on-stack and on-scope across sessions.

VIBE PROMPT 1 | Master context (paste first, every session)

```
You are a senior full-stack engineer pair-programming with me on KrishiSync AI, a
hackathon project for Challenge 1: Sustainable Agriculture & Resource Optimization.

PRODUCT
A web platform that connects Indian smallholder farmers to live market data and
grid status. Two engines:
1. Regional Anti-Saturation (anti-glut) Engine: warns farmers when too many
   farms within a 20 km radius plan the same crop, and recommends
   market-protected alternatives.
2. Smart Energy Engine: schedules irrigation for off-peak hours, supports
   PM-KUSUM solar pumps, pauses pumps when rain is forecast, tracks idle
   (vampire) power draw.
Plus a terrain-aware agro-advisory timeline (soil moisture, N-P-K, geomorphology).

USERS
Low-literacy smallholders on cheap Android phones and weak networks. Hindi and
English first, more vernacular languages later. UI must be icon-first, large tap
targets, color-coded (green / amber / red).

STACK (do not swap without asking)
Frontend: React.js (Vite), react-i18next, Leaflet, Recharts, PWA.
Backend: Python FastAPI. Database: PostgreSQL + PostGIS. Cache/queue: Redis.
Hardware: ESP32 + capacitive soil-moisture probes posting JSON over HTTPS.

RULES
- Build in small vertical slices. After each slice: list files changed, how to
  run it, and how to test it.
- Keep decision logic (scores, thresholds, schedules) in pure, unit-tested Python
  functions, separate from API routes.
- All thresholds and weights live in one config file, never hard-coded.
- No real farmer data. Use seeded demo data and say clearly when data is mocked.
- Ask me before adding any new dependency or changing the schema.
- Prefer boring, readable code over clever code. Add type hints and docstrings.

Reply "READY" and wait for my first task.
```

How to vibe-code this project well

- **One slice at a time.** Ask for a single endpoint or a single screen, run it, then move on.
- **Logic out of routes.** Scores and schedules are pure functions. That makes them testable and lets the data-science teammate own them.
- **Mock honestly.** Mandi prices and neighbour crop registrations will be seeded demo data. Say so on screen.
- **Commit after every green run.** If the AI breaks something, you can roll back in seconds.
- **Paste errors verbatim.** Give the full stack trace and the file, not a paraphrase.
- **Review before accepting.** Read every diff that touches the schema or the scoring logic.